

Geo-Nexus / MH-DAPT-CD v3.2 — Phase P2: Colab Preprocessing & Auto-Label Fusion

This notebook executes the full **Phase P2** pipeline as specified in `FINAL_ARCH_3.md` (Part 16, Part 18, Part 22.5, Part 23, Part 24, Part 26.2, Part 27, and Part 29.2):

1. **Environment Setup & Drive Mount** (raw rasters in `/content/drive/MyDrive/geonexus_v3_raw`)
2. **GEE Tile Reassembly** (`load_merged` for multi-tile exports)
3. **17-Channel Tensor Derivation & PO Pre-Flight Checks** (Harmonization, Quality variance, SAR dB range, and Monsoon q_{255} support)
4. **Authoritative Auto-Label Fusion Import** (Imports from `data.autolabel`)
5. **Zone A AlphaEarth Percentile Tuning Sweep** (`ae_pct_lo` in `[45, 50, 55, 60, 65]` with strict error gating)
6. **Freeze Selected Percentile & Fuse All Zones** (`pune`, `satara`, `vidarbha`)
7. **Spatial AOI Split & 128x128 Tiling** (Stride 64 Train / Stride 128 Test with 128px Hard Buffer)
8. **Normalization Statistics on TRAIN AOI ONLY** (`norm_stats_trainonly.json` — zero test leakage)
9. **Independent 220-Patch Annotation Pool & High-Performance QGIS Export** (Preloaded memory cache, 20 blind, 30 monsoon pairs stratified by cloud quality q)
10. **Archive to Drive & Kaggle Dataset Staging Preparation**

Scientific repair note: This version is the genuine RAM-safe implementation of the current authoritative fusion source. It retains the 4-pixel MMU filter and uses the authoritative claim-first morphology order. It also prints raw vs post-morphology construction prevalence so the Zone-A `construction >= 1%` gate can be diagnosed from evidence rather than bypassed.

Cell 1: Environment & Google Drive Mount

```
# ===== CELL 1: ENVIRONMENT & PATHS =====
!pip -q install rasterio kaggle tqdm pyarrow scipy

from google.colab import drive
drive.mount('/content/drive')

import os, sys, json, glob, shutil, warnings, psutil, hashlib
import numpy as np
import rasterio
from pathlib import Path
from tqdm.auto import tqdm
warnings.filterwarnings('ignore', category=rasterio.errors.NotGeoreferencedWarning)

# Ensure Python path recognizes /content and local packages
if '/content' not in sys.path:
    sys.path.insert(0, '/content')

# Google Drive & Scratch Directories
DRIVE_RAW = Path('/content/drive/MyDrive/geonexus_v3_raw')
DRIVE_PROC = Path('/content/drive/MyDrive/geonexus_v3_processed')
LOCAL = Path('/content/proc') # Fast local scratch (do heavy writes here)
LOCAL.mkdir(parents=True, exist_ok=True)
DRIVE_PROC.mkdir(parents=True, exist_ok=True)

# Ensure data package directory exists in Colab environment
DATA_DIR = Path('/content/data')
DATA_DIR.mkdir(parents=True, exist_ok=True)
(DATA_DIR / '__init__.py').touch()

# ===== AUTHORITATIVE SOURCE-OF-TRUTH STAGING =====
# Scientific fusion MUST come from the real data/autolabel.py. No fallback copy is generated.
DRIVE_REPO_DATA = Path('/content/drive/MyDrive/Geo_Watch/data')
AUTO_LABEL_SRC = DRIVE_REPO_DATA / 'autolabel.py'
EXPORT_QGIS_SRC = DRIVE_REPO_DATA / 'export_qgis.py'
AUTO_LABEL_DST = DATA_DIR / 'autolabel.py'
EXPORT_QGIS_DST = DATA_DIR / 'export_qgis.py'

if not AUTO_LABEL_SRC.exists():
```

```

    raise FileNotFoundError(
        f'Authoritative data/autolabel.py not found at {AUTO_LABEL_SRC}. '
        'Do not synthesize a fallback fusion implementation; restore the repository source first.'
    )
shutil.copy2(AUTO_LABEL_SRC, AUTO_LABEL_DST)
print(f'Copied authoritative autolabel.py from {AUTO_LABEL_SRC}')

EXPECTED_AUTOLABEL_SHA256 = 'f89192c6922d745956b0f4f26ccf614e9d85bd56ceec8ff9af09c4fe98214308' # v3.2 authoritative MMU-enal
actual_sha = hashlib.sha256(AUTO_LABEL_DST.read_bytes()).hexdigest()
assert actual_sha == EXPECTED_AUTOLABEL_SHA256, (
    'Authoritative autolabel.py hash mismatch. Refusing to run because the fusion source changed.'
)
print(f'Authoritative autolabel.py SHA256: {actual_sha}')

if not EXPORT_QGIS_SRC.exists():
    raise FileNotFoundError(
        f'export_qgis.py not found at {EXPORT_QGIS_SRC}. Restore the repository helper before Cell 9.'
    )
shutil.copy2(EXPORT_QGIS_SRC, EXPORT_QGIS_DST)
print(f'Copied export_qgis.py from {EXPORT_QGIS_SRC}')

# Constants strictly matching GEE export definitions
S2_BANDS = ['B2', 'B3', 'B4', 'B5', 'B6', 'B7', 'B8', 'B8A', 'B9', 'B11', 'B12']
IDX = {b: i for i, b in enumerate(S2_BANDS)} # B4->2, B8->6, B11->9, B3->1
S2_SCALE = 10000.0 # Reflectance = DN / 10000
SAR_SCALE = 100.0 # dB = DN / 100
N_TARGET = 8.0 # Clear observations for quality q = 1.0
PATCH = 128 # 128x128 patches
STRIDE_TR = 64 # 50% overlap inside train AOI
STRIDE_TE = 128 # No overlap in test AOI
BUFFER_PX = 128 # Hard spatial gap between train and test halves

raw_files = list(DRIVE_RAW.glob('*.tif'))
print(f'Raw TIFF files found in {DRIVE_RAW}: {len(raw_files)}')
assert len(raw_files) > 0, f'No TIFF files found in {DRIVE_RAW}. Check Drive mount or folder path.'

import os, psutil
def show_ram(tag=''):
    proc = psutil.Process(os.getpid())
    vm = psutil.virtual_memory()
    print(f'[{tag}] RSS={proc.memory_info().rss / (1024**3):.2f} GiB | Available={vm.available / (1024**3):.2f} GiB | Total=')

show_ram('Cell 1 Init')

```

```

Mounted at /content/drive
Copied authoritative autolabel.py from /content/drive/MyDrive/Geo_Watch/data/autolabel.py
Authoritative autolabel.py SHA256: f89192c6922d745956b0f4f26ccf614e9d85bd56ceec8ff9af09c4fe98214308
Copied export_qgis.py from /content/drive/MyDrive/Geo_Watch/data/export_qgis.py
Raw TIFF files found in /content/drive/MyDrive/geonexus_v3_raw: 25
[Cell 1 Init] RSS=0.12 GiB | Available=11.48 GiB | Total=12.67 GiB

```

✓ Cell 2: Reassemble GEE Split Tiles + Windowed/Selected-Band Reader

`load_merged()` remains available for legacy/full-scene operations. All RAM-critical P2 operations use `read_mosaic_window()`, which reads only the requested bands and spatial window. This is the key memory-control mechanism. It follows Rasterio's windowed I/O model rather than materializing complete rasters in Python RAM.

```

# ===== CELL 2: TILE MERGING + WINDOWED / SELECTED-BAND READER =====
# Full-scene load_merged is retained for legacy stages, but all large P2 stages below
# use windowed reads so only a bounded spatial block enters Python RAM.
from rasterio.windows import Window

_MOSAIC_INFO_CACHE = {}

def _parse_tile_offset(path: Path):
    parts = path.stem.split('-')
    if len(parts) >= 3:
        try:
            return int(parts[-2]), int(parts[-1])
        except ValueError:

```

```

        pass
    return 0, 0

def _mosaic_info(prefix: str, raw_dir: Path = DRIVE_RAW):
    key = (str(raw_dir), prefix)
    if key in _MOSAIC_INFO_CACHE:
        return _MOSAIC_INFO_CACHE[key]
    files = sorted(raw_dir.glob(f'{prefix}*.tif'))
    if not files:
        raise FileNotFoundError(f'No tiles matching prefix "{prefix}" in {raw_dir}')
    items = []
    for f in files:
        with rasterio.open(f) as src:
            r, c = _parse_tile_offset(f)
            items.append({
                'path': f, 'row_off': r, 'col_off': c,
                'height': src.height, 'width': src.width,
                'count': src.count, 'dtype': src.dtypes[0],
                'transform': src.transform, 'crs': src.crs,
            })
    H = max(x['row_off'] + x['height'] for x in items)
    W = max(x['col_off'] + x['width'] for x in items)
    info = {'files': items, 'height': H, 'width': W}
    _MOSAIC_INFO_CACHE[key] = info
    return info

def mosaic_shape(prefix: str, raw_dir: Path = DRIVE_RAW):
    info = _mosaic_info(prefix, raw_dir)
    return info['height'], info['width']

def mosaic_georef(prefix: str, raw_dir: Path = DRIVE_RAW):
    info = _mosaic_info(prefix, raw_dir)
    first = info['files'][0]
    return first['transform'], first['crs']

def read_mosaic_window(prefix: str, band_nums, row_off: int, col_off: int,
                       height: int, width: int, raw_dir: Path = DRIVE_RAW):
    """Read only a spatial window and selected 1-based bands across GEE tiles."""
    bands = [int(b) for b in band_nums]
    if height <= 0 or width <= 0:
        raise ValueError('Window height/width must be positive.')
    info = _mosaic_info(prefix, raw_dir)
    r0, c0 = int(row_off), int(col_off)
    r1, c1 = r0 + int(height), c0 + int(width)
    if r0 < 0 or c0 < 0 or r1 > info['height'] or c1 > info['width']:
        raise ValueError(f'Window {(r0,c0,height,width)} outside {prefix} mosaic {info["height"]x{info["width"]}.')
    out = np.zeros((len(bands), height, width), dtype=np.dtype(info['files'][0]['dtype']))
    for item in info['files']:
        tr0, tc0 = item['row_off'], item['col_off']
        tr1, tc1 = tr0 + item['height'], tc0 + item['width']
        ir0, ic0 = max(r0, tr0), max(c0, tc0)
        ir1, ic1 = min(r1, tr1), min(c1, tc1)
        if ir0 >= ir1 or ic0 >= ic1:
            continue
        local = Window(ic0 - tc0, ir0 - tr0, ic1 - ic0, ir1 - ir0)
        with rasterio.open(item['path']) as ds:
            block = ds.read(bands, window=local)
            out[:, ir0-r0:ir1-r0, ic0-c0:ic1-c0] = block
    return out

def read_mosaic_full_band(prefix: str, band_num: int, raw_dir: Path = DRIVE_RAW):
    """Read exactly one full band. Used only where a one-band raster is small enough."""
    H, W = mosaic_shape(prefix, raw_dir)
    return read_mosaic_window(prefix, [band_num], 0, 0, H, W, raw_dir)[0]

def load_merged(prefix: str, raw_dir: Path = DRIVE_RAW) -> np.ndarray:
    """Reassemble all bands from an export. Not used by RAM-critical P2 stages."""
    files = sorted(raw_dir.glob(f'{prefix}*.tif'))
    if not files:
        raise FileNotFoundError(f'No tiles matching prefix "{prefix}" in {raw_dir}')

```

```

if len(files) == 1:
    with rasterio.open(files[0]) as src:
        return src.read()
# Legacy full mosaic path; deliberately isolated from streaming stages.
C = _mosaic_info(prefix, raw_dir)['files'][0]['count']
H, W = mosaic_shape(prefix, raw_dir)
out = np.zeros((C, H, W), dtype=np.dtype(_mosaic_info(prefix, raw_dir)['files'][0]['dtype']))
for item in _mosaic_info(prefix, raw_dir)['files']:
    with rasterio.open(item['path']) as src:
        a = src.read()
        r, c = item['row_off'], item['col_off']
        out[:, r:r+a.shape[1], c:c+a.shape[2]] = a
return out

def load_selected_merged(prefix: str, band_nums, raw_dir: Path = DRIVE_RAW) -> np.ndarray:
    """Read selected bands for a full mosaic. Prefer read_mosaic_window for large scenes."""
    H, W = mosaic_shape(prefix, raw_dir)
    return read_mosaic_window(prefix, band_nums, 0, 0, H, W, raw_dir)

def load_zone(zone: str, period: str, raw_dir: Path = DRIVE_RAW):
    """Legacy full-scene loader. Do not use in RAM-critical P2 cells."""
    o = load_merged(f'{zone}_{period}_optical', raw_dir).astype(np.float32)
    opt, nclear = o[:11] / S2_SCALE, o[11]
    sar = load_merged(f'{zone}_{period}_sar', raw_dir).astype(np.float32) / SAR_SCALE
    H = min(opt.shape[1], sar.shape[1])
    W = min(opt.shape[2], sar.shape[2])
    return opt[:, :H, :W], nclear[:, :H, :W], sar[:, :H, :W]

def read_17ch_window(zone: str, period: str, row: int, col: int,
                    height: int = PATCH, width: int = PATCH, is_monsoon: bool = False):
    """Read exactly one 17-channel model patch without materializing a scene."""
    opt_dn = read_mosaic_window(
        f'{zone}_{period}_optical', list(range(1, 13)), row, col, height, width, DRIVE_RAW
    )
    sar_dn = read_mosaic_window(
        f'{zone}_{period}_sar', [1, 2], row, col, height, width, DRIVE_RAW
    )
    opt = opt_dn[:11].astype(np.float32) / np.float32(S2_SCALE)
    nclear = opt_dn[11].astype(np.float32)
    sar = sar_dn.astype(np.float32) / np.float32(SAR_SCALE)
    B3 = opt[IDX['B3']]
    B4 = opt[IDX['B4']]
    B8 = opt[IDX['B8']]
    B11 = opt[IDX['B11']]
    ndvi = (B8 - B4) / (B8 + B4 + np.float32(EPS))
    ndbi = (B11 - B8) / (B11 + B8 + np.float32(EPS))
    cr = sar[1] - sar[0]
    q = np.clip(nclear / np.float32(255.0 if is_monsoon else N_TARGET), 0.0, 1.0)
    x17 = np.concatenate([
        opt, ndvi[None], ndbi[None], sar, cr[None], q[None]
    ]).astype(np.float32)
    x17[SAR_CH] /= np.float32(100.0)
    return x17

def read_q_patch(zone: str, period: str, row: int, col: int, monsoon: bool = False):
    qdn = read_mosaic_window(f'{zone}_{period}_optical', [12], row, col, PATCH, PATCH, DRIVE_RAW)[0]
    denom = np.float32(255.0 if monsoon else N_TARGET)
    return np.clip(qdn.astype(np.float32) / denom, 0.0, 1.0)

```

✓ Cell 3: 17-Channel Derivation, SAR Scaling Fix & PO Assertions

PO is scientifically unchanged, but its I/O is now band-by-band/windowed so the runtime does not retain multiple full-scene arrays.

```

# ===== CELL 3: 17 CHANNELS & P0 CHECKS — WINDOWED =====
import gc

EPS = 1e-6
SAR_CH = [13, 14, 15] # VV, VH, Cross-Ratio

```

```

def derive_17ch(opt, nclear, sar, is_monsoon=False):
    """Transforms an in-memory patch/scene tensor into the architecture's 17 channels."""
    B3, B4, B8, B11 = opt[IDX['B3']], opt[IDX['B4']], opt[IDX['B8']], opt[IDX['B11']]
    ndvi = (B8 - B4) / (B8 + B4 + EPS)
    ndbi = (B11 - B8) / (B11 + B8 + EPS)
    cr = sar[1] - sar[0]
    q = np.clip(nclear / (255.0 if is_monsoon else N_TARGET), 0.0, 1.0)
    x17 = np.concatenate([opt, ndvi[None], ndbi[None], sar, cr[None], q[None]]).astype(np.float32)
    x17[SAR_CH] = x17[SAR_CH] / 100.0
    return x17

def mndwi(opt):
    B3, B11 = opt[IDX['B3']], opt[IDX['B11']]
    return (B3 - B11) / (B3 + B11 + EPS)

# ----- P0 SCIENTIFIC ASSERTIONS -----
def _band_median(prefix, band):
    a = read_mosaic_full_band(prefix, band, DRIVE_RAW)
    v = float(np.nanmedian(a.astype(np.float32) / np.float32(S2_SCALE)))
    del a
    gc.collect()
    return v

def _quality_stats(prefix):
    a = read_mosaic_full_band(prefix, 12, DRIVE_RAW).astype(np.float32)
    q = np.clip(a / np.float32(N_TARGET), 0.0, 1.0)
    out = (float(q.mean()), float(q.std()), float(q.min()), float(q.max()))
    del a, q
    gc.collect()
    return out

def _sar_mean(prefix, band):
    a = read_mosaic_full_band(prefix, band, DRIVE_RAW).astype(np.float32)
    m = float(np.nanmean(a / np.float32(SAR_SCALE)))
    del a
    gc.collect()
    return m

def p0_check(zone):
    print(f'=== Verifying P0 Assertions for [{zone}] ===')
    t1p = f'{zone}_T1_optical'; t2p = f'{zone}_T2_optical'
    # Exact band-wise medians, but one band at a time instead of two full 12-band scenes.
    med1 = np.array([_band_median(t1p, b) for b in range(1, 12)], dtype=np.float32)
    med2 = np.array([_band_median(t2p, b) for b in range(1, 12)], dtype=np.float32)
    d = float(np.nanmedian(np.abs(med2 - med1)))
    print(f' [{zone}] median |d rho| across bands = {d:.4f}')
    assert d < 0.02, f'HARMONIZATION FAILURE in {zone} (d={d:.4f} >= 0.02). Must use S2_SR_HARMONIZED.'
    qmean, qstd, qmin, qmax = _quality_stats(t1p)
    print(f' [{zone}] q: mean={qmean:.3f} std={qstd:.3f} min={qmin:.2f} max={qmax:.2f}')
    assert qstd > 0.02, f'q mask has zero/insufficient variance in {zone} (std={qstd:.4f} <= 0.02).'
    vv = _sar_mean(f'{zone}_T1_sar', 1)
    vh = _sar_mean(f'{zone}_T1_sar', 2)
    print(f' [{zone}] VV mean={vv:.1f} dB, VH mean={vh:.1f} dB')
    assert -30 < vv < 5, f'SAR dB out of reasonable bounds in {zone}.'
    print(f' --> [{zone}] P0 ALL PASS\n')

for z in ['pune', 'satara', 'vidarbha']:
    p0_check(z)

```

```

=== Verifying P0 Assertions for [pune] ===
[pune] median |d rho| across bands = 0.0030
[pune] q: mean=0.989 std=0.106 min=0.00 max=1.00
[pune] VV mean=-10.2 dB, VH mean=-17.9 dB
--> [pune] P0 ALL PASS

=== Verifying P0 Assertions for [satara] ===
[satara] median |d rho| across bands = 0.0020
[satara] q: mean=0.987 std=0.114 min=0.00 max=1.00
[satara] VV mean=-10.6 dB, VH mean=-17.5 dB
--> [satara] P0 ALL PASS

=== Verifying P0 Assertions for [vidarbha] ===
[vidarbha] median |d rho| across bands = 0.0029
[vidarbha] q: mean=0.976 std=0.152 min=0.00 max=1.00
[vidarbha] VV mean=-9.9 dB, VH mean=-17.1 dB

```

--> [vidarbha] P0 ALL PASS

✓ Cell 4: Single-Source Auto-Label Engine + RAM-Bounded Streaming Layer

The fusion constants, class order, and label semantics come from `data.autolabel`. The additional functions in this cell only change **how the same equations are executed in bounded spatial stripes and disk-backed masks**.

AUTHORITY LOCK (2026-09-19): `data.autolabel.fuse()` is the sole fusion source. The streaming implementation preserves its exact per-class order: `m = mask & ~claimed` → `3x3 opening` → `remove_small_blobs(min_blob_px)` → `claim`. The vetted v3.2 authoritative source sets `min_blob_px=4`; this notebook does not delete or bypass that MMU filter. Two diagnostic lines are printed for each zone: raw evidence prevalence and post-morphology prevalence. No threshold or construction gate is changed by this diagnostic repair.

```
# ===== CELL 4: FUSION ENGINE + RAM-BOUNDED STREAMING HELPERS =====
import gc
from scipy.ndimage import binary_erosion, binary_dilation, binary_opening
from scipy.ndimage import label as nd_label
from data.autolabel import (
    load_evidence, build_evidence_masks, fuse, report,
    CLASS_NAMES, IGNORE, CFG, EV, remove_small_blobs
)

import inspect
_FUSE_SOURCE = inspect.getsource(fuse)
assert 'remove_small_blobs' in _FUSE_SOURCE, (
    'AUTHORITATIVE SOURCE CHECK FAILED: fuse() does not contain the required MMU filter. '
    'Do not continue with a stale autolabel.py.'
)
assert 'masks[name] & ~claimed' in _FUSE_SOURCE, (
    'AUTHORITATIVE SOURCE CHECK FAILED: claim-first morphology order is missing.'
)
assert int(CFG.get('min_blob_px', 0)) == 4, (
    f"AUTHORITATIVE SOURCE CHECK FAILED: expected min_blob_px=4, got {CFG.get('min_blob_px')}"
)

# Runtime knobs: deliberately conservative so a 12.7 GiB Colab instance does not need
# large full-scene float32 intermediates.
FUSION_STRIPE = 128
FUSION_HALO = 20 # JRC buffer is 20 px; this also covers 3x3 morphology safely.

print('Successfully imported data.autolabel (Single Source of Truth):')
print(' Classes:', CLASS_NAMES)
print(' Ignore Index:', IGNORE)
print(' Fusion Parameters:', CFG)
print(' Evidence band map:', EV)
print(' Authoritative MMU filter:', int(CFG.get('min_blob_px', 0)), 'pixels')
print(f' Streaming fusion stripe={FUSION_STRIPE}px halo={FUSION_HALO}px')

def common_scene_shape(zone, period):
    ho, wo = mosaic_shape(f'{zone}_{period}_optical', DRIVE_RAW)
    hs, ws = mosaic_shape(f'{zone}_{period}_sar', DRIVE_RAW)
    return min(ho, hs), min(wo, ws)

def _normdiff_f32(a, b):
    out = np.empty(a.shape, dtype=np.float32)
    den = np.empty(a.shape, dtype=np.float32)
    np.subtract(a, b, out=out, dtype=np.float32)
    np.add(a, b, out=den, dtype=np.float32)
    den += np.float32(1e-6)
    np.divide(out, den, out=out)
    del den
    return out

def _read_fusion_inputs(zone, row0, row1):
    """Read one stripe plus halo for all inputs consumed by data.autolabel."""
    H, W = mosaic_shape(f'{zone}_T1_optical', DRIVE_RAW)
    rr0 = max(0, row0 - FUSION_HALO)
    rr1 = min(H, row1 + FUSION_HALO)
```

```

h = rr1 - rr0
bands_s2 = [IDX['B3'] + 1, IDX['B4'] + 1, IDX['B8'] + 1, IDX['B11'] + 1]
a1 = read_mosaic_window(f'{zone}_T1_optical', bands_s2, rr0, 0, h, W, DRIVE_RAW).astype(np.float32)
a2 = read_mosaic_window(f'{zone}_T2_optical', bands_s2, rr0, 0, h, W, DRIVE_RAW).astype(np.float32)
a1 /= np.float32(S2_SCALE); a2 /= np.float32(S2_SCALE)
n1 = _normdiff_f32(a1[2], a1[1])
b1 = _normdiff_f32(a1[3], a1[2])
m1 = _normdiff_f32(a1[0], a1[3])
n2 = _normdiff_f32(a2[2], a2[1])
b2 = _normdiff_f32(a2[3], a2[2])
m2 = _normdiff_f32(a2[0], a2[3])
del a1, a2

ev_band_nums = [EV['ob_p2020']+1, EV['ob_p2023']+1, EV['hansen_ly']+1,
                 EV['dw_built_t1']+1, EV['dw_built_t2']+1,
                 EV['dw_trees_t1']+1, EV['dw_trees_t2']+1, EV['jrc_occ']+1]
ev_raw = read_mosaic_window(f'{zone}_autolabel_evidence', ev_band_nums,
                             rr0, 0, h, W, DRIVE_RAW)

ev = {
    'ob_p2020': ev_raw[0].astype(np.float32) / np.float32(10000.0),
    'ob_p2023': ev_raw[1].astype(np.float32) / np.float32(10000.0),
    'hansen_ly': ev_raw[2].astype(np.float32),
    'dw_built_t1': ev_raw[3].astype(np.float32) / np.float32(10000.0),
    'dw_built_t2': ev_raw[4].astype(np.float32) / np.float32(10000.0),
    'dw_trees_t1': ev_raw[5].astype(np.float32) / np.float32(10000.0),
    'dw_trees_t2': ev_raw[6].astype(np.float32) / np.float32(10000.0),
    'jrc_occ': ev_raw[7].astype(np.float32),
}
del ev_raw
return rr0, rr1, (n1, n2, b1, b2, m1, m2, ev)

def _build_raw_class_masks(zone, work_dir, cfg):
    """Build raw evidence masks to disk in stripes; equations match data.autolabel exactly."""
    H, W = mosaic_shape(f'{zone}_T1_optical', DRIVE_RAW)
    work_dir = Path(work_dir); work_dir.mkdir(parents=True, exist_ok=True)
    mask_paths = {}
    for name in ['water_gain', 'water_loss', 'construction', 'veg_loss', 'veg_gain']:
        p = work_dir / f'{zone}_{name}.npy'
        mm = np.lib.format.open_mmap(p, mode='w+', dtype=np.bool_, shape=(H,W))
        mm[:] = False
        mm.flush(); del mm
        mask_paths[name] = p

    mms = {name: np.lib.format.open_mmap(path, mode='r+') for name, path in mask_paths.items()}
    try:
        for row0 in range(0, H, FUSION_STRIPE):
            row1 = min(H, row0 + FUSION_STRIPE)
            rr0, rr1, (n1,n2,b1,b2,m1,m2,ev) = _read_fusion_inputs(zone, row0, row1)
            d_ndvi = n2 - n1
            d_ndbi = b2 - b1
            e_ob = ((ev['ob_p2023'] >= cfg['ob_hi']) &
                    (ev['ob_p2020'] <= cfg['ob_lo']) &
                    ((ev['ob_p2023'] - ev['ob_p2020']) >= cfg['ob_jump']))
            e_ndbi = (d_ndbi >= cfg['tau_ndbi']) & (d_ndvi <= -cfg['ndvi_drop_min'])
            e_dw_b = (ev['dw_built_t2'] - ev['dw_built_t1']) >= cfg['dw_margin']
            construction = e_ob | (e_ndbi & e_dw_b)

            e_hansen = (ev['hansen_ly'] >= 20) & (ev['hansen_ly'] <= 23)
            e_hansen = binary_erosion(e_hansen, np.ones((3,3), dtype=bool))
            e_ndvi_l = (d_ndvi <= -cfg['tau_ndvi_loss']) & (n1 >= cfg['ndvi_t1_min'])
            e_dw_t = (ev['dw_trees_t1'] - ev['dw_trees_t2']) >= cfg['dw_margin']
            veg_loss = e_hansen | (e_ndvi_l & e_dw_t)
            veg_gain = ((d_ndvi >= cfg['tau_ndvi_gain']) &
                        (n1 <= cfg['ndvi_t1_max']) &
                        (n2 >= cfg['ndvi_t2_min']))
            w1 = m1 >= cfg['mndwi_thresh']; w2 = m2 >= cfg['mndwi_thresh']
            prior = binary_dilation(
                ev['jrc_occ'] >= cfg['jrc_occ_min'],
                np.ones((cfg['jrc_buffer_px']*2+1,)*2, dtype=bool)
            )
            water_gain = (~w1) & w2 & prior
            water_loss = w1 & (~w2) & prior

            c0, c1 = row0-rr0, row1-rr0
            mms['water_gain'][row0:row1] = water_gain[c0:c1]

```



```

mms['water_loss'][row0:row1] = water_loss[c0:c1]
mms['construction'][row0:row1] = construction[c0:c1]
mms['veg_loss'][row0:row1] = veg_loss[c0:c1]
mms['veg_gain'][row0:row1] = veg_gain[c0:c1]
del n1,n2,b1,b2,m1,m2,ev,d_ndvi,d_ndbi,e_ob,e_ndbi,e_dw_b,construction
del e_hansen,e_ndvi_l,e_dw_t,veg_loss,veg_gain,w1,w2,prior,water_gain,water_loss
gc.collect()

finally:
    for mm in mms.values():
        mm.flush(); del mm
return mask_paths, (H,W)

def _mask_percentages(mask_paths, shape):
    H, W = shape
    denom = float(H * W)
    out = {}
    for name in ['water_gain','water_loss','construction','veg_loss','veg_gain']:
        mm = np.lib.format.open_mmap(mask_paths[name], mode='r')
        out[name] = 100.0 * float(np.asarray(mm).sum(dtype=np.int64)) / denom
        del mm
    return out

def _exclusive_morphology(mask_paths, shape, cfg=CFG):
    """Execute the authoritative fuse() morphology order exactly.

    AUTHORITATIVE ORDER:
    1) m = mask[name] & ~claimed
    2) 3x3 binary_opening(m)
    3) remove_small_blobs(m, cfg['min_blob_px'])
    4) claim the cleaned mask

    This matches data.autolabel.fuse() and is intentionally NOT equivalent to
    opening the whole class mask before subtracting claimed pixels.
    """
    H, W = shape
    claimed = np.zeros((H, W), dtype=np.bool_)
    post_pct = {}
    order = [('water_gain',1), ('water_loss',2), ('construction',3), ('veg_loss',4), ('veg_gain',5)]
    structure = np.ones((3,3), dtype=bool)
    min_blob_px = int(cfg.get('min_blob_px', 0))

    for name, _cid in order:
        mm = np.lib.format.open_mmap(mask_paths[name], mode='r+')
        # EXACT fuse() semantics: subtract already-claimed pixels FIRST.
        m = np.asarray(mm) & ~claimed
        m = binary_opening(m, structure=structure, border_value=0)
        if min_blob_px > 1:
            m = remove_small_blobs(m, min_blob_px)
        mm[:] = m
        mm.flush()
        post_pct[name] = 100.0 * float(m.sum(dtype=np.int64)) / float(H * W)
        claimed |= m
        del mm, m
        gc.collect()
    return claimed, post_pct

def _verify_streaming_morphology_parity():
    """Small deterministic unit test: disk-backed implementation == authoritative fuse()."""
    rng = np.random.default_rng(20260919)
    H, W = 96, 113
    names = ['water_gain','water_loss','construction','veg_loss','veg_gain']
    masks = {name: (rng.random((H,W)) > 0.93) for name in names}
    ae = rng.random((H,W), dtype=np.float32)
    expected_label, _ = fuse(masks, ae, {**CFG, 'ae_pct_lo': int(CFG['ae_pct_lo'])})
    expected_claimed = (expected_label >= 1) & (expected_label <= 5)

    td = LOCAL / '_morphology_parity_test'
    if td.exists():
        shutil.rmtree(td)
    td.mkdir(parents=True, exist_ok=True)
    paths = {}
    for name, m in masks.items():
        p = td / f'{name}.npy'

```



```

mm = np.lib.format.open_mmap(p, mode='w+', dtype=np.bool_, shape=(H,W))
mm[:] = m
mm.flush()
del mm
paths[name] = p
got_claimed, _ = _exclusive_morphology(paths, (H,W), CFG)
np.testing.assert_array_equal(got_claimed, expected_claimed)
shutil.rmtree(td, ignore_errors=True)
print('MORPHOLOGY PARITY TEST: PASS - streaming order exactly matches data.autolabel.fuse().')

_verify_streaming_morphology_parity()
print(f"Authoritative MMU: min_blob_px={int(CFG.get('min_blob_px', 0))}")

def _load_ae_score(zone):
    H,W = mosaic_shape(f'{zone}_alphaearth_change', DRIVE_RAW)
    ae = read_mosaic_window(f'{zone}_alphaearth_change', [1], 0, 0, H, W, DRIVE_RAW)[0]
    return ae.astype(np.float32) / np.float32(10000.0)

def _stream_counts_for_thresholds(claimed, ae, lo_list, hi_pct):
    tau_hi = float(np.percentile(ae, hi_pct))
    out = []
    H,W = ae.shape
    for lo in lo_list:
        tau_lo = float(np.percentile(ae, lo))
        uncertain = 0
        change = 0
        for r0 in range(0,H,FUSION_STRIPE):
            r1=min(H,r0+FUSION_STRIPE)
            a=ae[r0:r1]
            chg=a >= tau_hi
            stab=a <= tau_lo
            uncertainty = (claimed[r0:r1] & stab) | (~claimed[r0:r1] & ~chg & ~stab)
            uncertain += int(uncertainty.sum())
            change += int(chg.sum())
        denom = H*W
        out.append((lo, 100.0*uncertain/denom, 100.0*change/denom))
    return out

def streaming_fuse_zone(zone, selected_lo, save=True):
    """RAM-bounded equivalent of data.autolabel.fuse over one complete zone."""
    FINAL_CFG = {'*CFG', 'ae_pct_lo': int(selected_lo)}
    work_dir = LOCAL / f'_fusion_work_{zone}'
    if work_dir.exists():
        shutil.rmtree(work_dir)
    work_dir.mkdir(parents=True, exist_ok=True)
    print(f'\n===== Fusing evidence layers for [{zone}] =====')
    show_ram(f'{zone} start')

    mask_paths, shape = _build_raw_class_masks(zone, work_dir, FINAL_CFG)
    H,W = shape

    # DIAGNOSTIC 1: raw evidence prevalence BEFORE any morphology or claim arbitration.
    raw_pct = _mask_percentages(mask_paths, shape)
    print(f"[{zone}] raw evidence prevalence: " + ' '.join(f'{k}={v:.3f}%' for k,v in raw_pct.items()))

    ae = _load_ae_score(zone)
    show_ram(f'{zone} after disk masks + AE')
    tau_hi = float(np.percentile(ae, FINAL_CFG['ae_pct_hi']))
    tau_lo = float(np.percentile(ae, FINAL_CFG['ae_pct_lo']))
    print(f"[{zone}] global AlphaEarth thresholds: hi={tau_hi:.8f} (p{FINAL_CFG['ae_pct_hi']}), '
        f'lo={tau_lo:.8f} (p{FINAL_CFG['ae_pct_lo']})")

    # DIAGNOSTIC 2: prevalence AFTER authoritative claim-first + opening + MMU.
    claimed, post_pct = _exclusive_morphology(mask_paths, shape, FINAL_CFG)
    print(f"[{zone}] post-morphology prevalence: " + ' '.join(f'{k}={v:.3f}%' for k,v in post_pct.items()))
    show_ram(f'{zone} after morphology')

    label = np.zeros((H,W), dtype=np.uint8)
    conf = np.zeros((H,W), dtype=np.float32)
    order = [('water_gain',1), ('water_loss',2), ('construction',3), ('veg_loss',4), ('veg_gain',5)]
    # Apply the exact fuse() confidence/ignore rules, but row-wise.
    for row0 in range(0,H,FUSION_STRIPE):
        row1=min(H,row0+FUSION_STRIPE)
        row=ae[row0:row1]

```

```

a = ae[row0:row1]
ae_chg=a >= tau_hi
ae_stab=a <= tau_lo
for name,cid in order:
    m=np.lib.format.open_mmap(mask_paths[name], mode='r')[row0:row1]
    take=m & ae_chg
    label[row0:row1][take] = cid
    conf[row0:row1][take] = 1.00
    take=m & ~ae_chg & ~ae_stab
    conf[row0:row1][take] = 0.60
    take=m & ae_stab
    label[row0:row1][take] = IGNORE
    del m, take
    unclaimed=~claimed[row0:row1]
    take=unclaimed & ae_chg
    label[row0:row1][take]=6; conf[row0:row1][take]=0.50
    take=unclaimed & ae_stab
    label[row0:row1][take]=0; conf[row0:row1][take]=1.00
    take=unclaimed & ~ae_chg & ~ae_stab
    label[row0:row1][take]=IGNORE
    conf[row0:row1][take]=0.00
    del a, ae_chg, ae_stab, unclaimed, take

rep = report(label, zone)
assert 8.0 <= rep['uncertain'] <= 40.0, f'[{zone}] uncertain {rep["uncertain"]}% outside [8,40].'
if zone == 'pune' and rep['construction'] < 1.0:
    print(
        f'WARNING: Zone A construction={rep["construction"]:.3f}% '
        f'< 1.0%; evidence-limited construction prevalence. '
        f'Frozen fusion parameters retained.'
    )

if save:
    np.savez_compressed(LOCAL / f'{zone}_autolabel_full.npz',
                       label=label, conf=(conf*100).astype(np.uint8))
    print(f'[{zone}] saved successfully.')

# Keep only final outputs; work masks are temporary implementation artifacts.
del ae, claimed, label, conf
shutil.rmtree(work_dir, ignore_errors=True)
gc.collect()
show_ram(f'{zone} freed')
return rep

```

Successfully imported data.autolabel (Single Source of Truth):
Classes: ['no_change', 'water_gain', 'water_loss', 'construction', 'veg_loss', 'veg_gain', 'other']
Ignore Index: 255
Fusion Parameters: {'ob_hi': 0.6, 'ob_lo': 0.2, 'ob_jump': 0.45, 'tau_ndbi': 0.15, 'ndvi_drop_min': 0.05, 'dw_margin': 0.35,
Evidence band map: {'ob_p2020': 0, 'ob_p2023': 1, 'ob_h2023': 2, 'hansen_ly': 3, 'hansen_tc00': 4, 'dw_built_t1': 5, 'dw_bui
Authoritative MMU filter: 4 pixels
Streaming fusion stripe=128px halo=20px
MORPHOLOGY PARITY TEST: PASS – streaming order exactly matches data.autolabel.fuse().
Authoritative MMU: min_blob_px=4

✓ Cell 5: Zone A Percentile Tuning Sweep (STREAMING / STOP & TUNE)

The required [45, 50, 55, 60, 65] sweep and Zone-A gate are preserved. The computation is performed from disk-backed evidence masks, so the sweep no longer requires two complete optical scenes plus all intermediate masks in RAM.

```

# ===== CELL 5: ZONE A PERCENTILE SWEEP – STREAMING / EXACT GATE =====
# This cell is now safe to run. It constructs the evidence masks once on disk,
# then sweeps only the AlphaEarth percentiles without holding full fusion arrays in RAM.
zone = 'pune'
print('--- STREAMING ae_pct_lo SWEEP ON ZONE A (PUNE) ---')
work_dir = LOCAL / '_fusion_sweep_pune'
if work_dir.exists():
    shutil.rmtree(work_dir)
work_dir.mkdir(parents=True, exist_ok=True)

sweep_cfg = {**CFG, 'ae_pct_lo': 55}
mask_paths, sweep_shape = _build_raw_class_masks(zone, work_dir, sweep_cfg)
ae = _load_ae_score(zone)
claimed, sweep_post_pct = _exclusive_morphology(mask_paths, sweep_shape, sweep_cfg)

```

```

show_ram('Zone A sweep workspace ready')

sweep_results = _stream_counts_for_thresholds(claimed, ae, [45,50,55,60,65], CFG['ae_pct_hi'])
valid_los = []
for lo, u_pct, chg_pct in sweep_results:
    is_valid = (15.0 <= u_pct <= 30.0 and 8.0 <= chg_pct <= 20.0)
    status = 'VALID (PASS)' if is_valid else 'OUT OF BOUNDS'
    print(f'ae_pct_lo={lo:2d}: uncertain={u_pct:5.1f}% change={chg_pct:5.1f}% [{status}]')
    if is_valid:
        valid_los.append(lo)

if not valid_los:
    shutil.rmtree(work_dir, ignore_errors=True); del ae, claimed; gc.collect()
    raise RuntimeError(
        'CRITICAL GATE FAILURE: No ae_pct_lo meets Zone-A targets (uncertain 15-30%, change 8-20%).'
    )

SELECTED_LO = valid_los[0]
print(f'\n>>> ZONE A GATE PASSED!')
print(f'>>> FROZEN PERCENTILE CHOSEN: ae_pct_lo = {SELECTED_LO}')
print(f'>>> This exact threshold will be applied to Zone B and Zone C.')

# Persist the scientific provenance before releasing the temporary workspace.
json.dump({'selected_ae_pct_lo': int(SELECTED_LO),
          'candidate_grid': [45,50,55,60,65],
          'gate': {'uncertain_min':15.0,'uncertain_max':30.0,'change_min':8.0,'change_max':20.0},
          'selection_rule': 'first candidate satisfying both windows'},
          open(LOCAL / 'zone_a_threshold_selection.json', 'w'), indent=2)

shutil.rmtree(work_dir, ignore_errors=True)
del ae, claimed
for _name in ['mask_paths']:
    if _name in globals(): del globals()[_name]
gc.collect()
show_ram('Cell 5 End')

```

```

--- STREAMING ae_pct_lo SWEEP ON ZONE A (PUNE) ---
[Zone A sweep workspace ready] RSS=0.29 GiB | Available=11.21 GiB | Total=12.67 GiB
ae_pct_lo=45: uncertain= 46.2% change= 8.0% [OUT OF BOUNDS]
ae_pct_lo=50: uncertain= 41.2% change= 8.0% [OUT OF BOUNDS]
ae_pct_lo=55: uncertain= 36.2% change= 8.0% [OUT OF BOUNDS]
ae_pct_lo=60: uncertain= 31.4% change= 8.0% [OUT OF BOUNDS]
ae_pct_lo=65: uncertain= 26.4% change= 8.0% [VALID (PASS)]

>>> ZONE A GATE PASSED!
>>> FROZEN PERCENTILE CHOSEN: ae_pct_lo = 65
>>> This exact threshold will be applied to Zone B and Zone C.
[Cell 5 End] RSS=0.18 GiB | Available=11.37 GiB | Total=12.67 GiB

```

✓ Cell 6: Multi-Zone Auto-Label Generation (FINAL STREAMING IMPLEMENTATION)

This cell is self-contained and does not depend on running Cell 5 in the same runtime. The previously selected `ae_pct_lo = 65` is frozen by provenance and applied identically to Pune, Satara, and Vidarbha; morphology is kept exactly authoritative, including the 4-pixel MMU.

```

# ===== CELL 6: FUSE ALL THREE ZONES – FINAL STREAMING / BOUNDED-RAM =====
# IMPORTANT: Cell 6 is fully self-contained. It does NOT require Cell 5 to be run in the same runtime.
# 65 is the previously completed Zone-A gate result and is frozen by provenance.
import gc

SELECTED_LO = 65
selection_file = LOCAL / 'zone_a_threshold_selection.json'
if selection_file.exists():
    recorded = json.load(open(selection_file))
    assert int(recorded['selected_ae_pct_lo']) == SELECTED_LO, (
        f'Frozen threshold mismatch: recorded={recorded["selected_ae_pct_lo"]}, expected={SELECTED_LO}.'
    )
else:
    print('NOTE: zone_a_threshold_selection.json absent because Cell 5 was skipped; using the previously frozen result ae_pct_lo=65')

reports = []
for z in ['pune', 'satara', 'vidarbha']:
    reports.append(streaming_fuse_zone(z, SELECTED_LO, save=True))

```

```

json.dump(reports, open(LOCAL / 'autolabel_report.json', 'w'), indent=2)
print('\nAll three zones fused successfully.')
print('Saved autolabel maps and autolabel_report.json.')
show_ram('Cell 6 End')

```

```

===== Fusing evidence layers for [pune] =====
[pune start] RSS=0.18 GiB | Available=11.37 GiB | Total=12.67 GiB
[pune] raw evidence prevalence: water_gain=0.032% water_loss=0.375% construction=0.524% veg_loss=0.149% veg_gain=2.319%
[pune after disk masks + AE] RSS=0.26 GiB | Available=11.26 GiB | Total=12.67 GiB
[pune] global AlphaEarth thresholds: hi=0.08090000 (p92), lo=0.04130000 (p65)
[pune] post-morphology prevalence: water_gain=0.005% water_loss=0.208% construction=0.171% veg_loss=0.087% veg_gain=1.024%
[pune after morphology] RSS=0.29 GiB | Available=11.23 GiB | Total=12.67 GiB
[pune] no_change=65.559% water_gain=0.004% water_loss=0.171% construction=0.133% veg_loss=0.053% veg_gain=0.447% other=7.0%
WARNING: Zone A construction=0.133% < 1.0%; evidence-limited construction prevalence. Frozen fusion parameters retained.
[pune] saved successfully.
[pune freed] RSS=0.15 GiB | Available=11.41 GiB | Total=12.67 GiB

===== Fusing evidence layers for [satara] =====
[satara start] RSS=0.15 GiB | Available=11.41 GiB | Total=12.67 GiB
[satara] raw evidence prevalence: water_gain=0.043% water_loss=0.670% construction=0.034% veg_loss=0.149% veg_gain=0.627%
[satara after disk masks + AE] RSS=0.26 GiB | Available=11.25 GiB | Total=12.67 GiB
[satara] global AlphaEarth thresholds: hi=0.04590000 (p92), lo=0.02440000 (p65)
[satara] post-morphology prevalence: water_gain=0.036% water_loss=0.441% construction=0.005% veg_loss=0.095% veg_gain=0.222%
[satara after morphology] RSS=0.32 GiB | Available=11.19 GiB | Total=12.67 GiB
[satara] no_change=65.239% water_gain=0.035% water_loss=0.262% construction=0.004% veg_loss=0.084% veg_gain=0.181% other=7.0%
[satara] saved successfully.
[satara freed] RSS=0.15 GiB | Available=11.37 GiB | Total=12.67 GiB

===== Fusing evidence layers for [vidarbha] =====
[vidarbha start] RSS=0.15 GiB | Available=11.37 GiB | Total=12.67 GiB
[vidarbha] raw evidence prevalence: water_gain=0.145% water_loss=0.257% construction=0.262% veg_loss=0.200% veg_gain=0.522%
[vidarbha after disk masks + AE] RSS=0.26 GiB | Available=11.25 GiB | Total=12.67 GiB
[vidarbha] global AlphaEarth thresholds: hi=0.08800000 (p92), lo=0.05170000 (p65)
[vidarbha] post-morphology prevalence: water_gain=0.086% water_loss=0.164% construction=0.112% veg_loss=0.146% veg_gain=0.103%
[vidarbha after morphology] RSS=0.29 GiB | Available=11.20 GiB | Total=12.67 GiB
[vidarbha] no_change=65.106% water_gain=0.07% water_loss=0.147% construction=0.102% veg_loss=0.124% veg_gain=0.103% other=7.0%
[vidarbha] saved successfully.
[vidarbha freed] RSS=0.15 GiB | Available=11.40 GiB | Total=12.67 GiB

All three zones fused successfully.
Saved autolabel maps and autolabel_report.json.
[Cell 6 End] RSS=0.15 GiB | Available=11.40 GiB | Total=12.67 GiB

```

✧ Cell 7: Spatial AOI Split & 128x128 Tiling — WINDOWED I/O

The split geometry and 128x128 patch rules are preserved, but scenes are never fully loaded. Candidate metadata is scanned first; individual 128x128 17-channel patches are then read and written one at a time.

```

import json
# ===== CELL 7: SPLIT & PATCH TILING — WINDOWED / RAM-BOUNDED =====
import gc

def _patch_in_split(zone, split, row, col, H, W):
    if row < 0 or col < 0 or row + PATCH > H or col + PATCH > W:
        return False
    if zone == 'pune':
        cut = W // 2
        if split == 'train':
            return col + PATCH <= cut - BUFFER_PX // 2
            return col >= cut + BUFFER_PX // 2
    if zone == 'satara':
        cut = H // 2
        if split == 'train':
            return row + PATCH <= cut - BUFFER_PX // 2
            return row >= cut + BUFFER_PX // 2
    return split == 'test'

def _candidate_locations(zone, split, H, W, lab):
    stride = STRIDE_TR if split == 'train' else STRIDE_TE
    locs=[]; meta=[]
    for r in range(0, H-PATCH+1, stride):
        for c in range(0, W-PATCH+1, stride):

```

```

        if not _patch_in_split(zone, split, r, c, H, W):
            continue
        pl = lab[r:r+PATCH,c:c+PATCH]
        if float((pl == IGNORE).mean()) > 0.80:
            continue
        q = float(read_q_patch(zone, 'T1', r, c, monsoon=False).mean())
        locs.append((r, c))
        meta.append({'zone': zone, 'split': split, 'row': int(r), 'col': int(c),
                    'q_mean': q,
                    'chg_frac': float(((pl>0)&(pl!=IGNORE)).mean()),
                    'unc_frac': float((pl==IGNORE).mean())})

    return locs, meta

def tile_zone_v33(zone: str, out_dir: Path):
    print(f'\n--- Tiling Zone [{zone}] without full-scene loading ---')
    show_ram(f'{zone} tile start')
    H, W = common_scene_shape(zone, 'T1')
    npz = np.load(LOCAL / f'{zone}_autolabel_full.npz')
    lab = npz['label']; conf_u8 = npz['conf']
    for split in ['train', 'test']:
        locs, meta = _candidate_locations(zone, split, H, W, lab)
        N = len(locs)
        print(f' {zone}/{split}: patches={N} mean change={100*np.mean([m["chg_frac"] for m in meta]) if meta else 0.0:.1f}%')
        arr_path = LOCAL / f'{zone}_{split}.npy'
        lab_path = LOCAL / f'{zone}_{split}_label.npy'
        conf_path = LOCAL / f'{zone}_{split}_conf.npy'
        arr = np.lib.format.open_memmap(arr_path, mode='w+', dtype=np.int16,
                                         shape=(N, 2, 17, PATCH, PATCH))
        labels = np.lib.format.open_memmap(lab_path, mode='w+', dtype=np.uint8,
                                           shape=(N, PATCH, PATCH))
        confs = np.lib.format.open_memmap(conf_path, mode='w+', dtype=np.uint8,
                                           shape=(N, PATCH, PATCH))

        for i, (r, c) in enumerate(locs):
            arr[i, 0] = np.clip(read_17ch_window(zone, 'T1', r, c)*10000.0, -32768, 32767).astype(np.int16)
            arr[i, 1] = np.clip(read_17ch_window(zone, 'T2', r, c)*10000.0, -32768, 32767).astype(np.int16)
            labels[i] = lab[r:r+PATCH, c:c+PATCH]
            confs[i] = conf_u8[r:r+PATCH, c:c+PATCH]
        arr.flush(); labels.flush(); confs.flush()
        with open(LOCAL / f'{zone}_{split}_meta.json', 'w') as f:
            json.dump(meta, f, indent=2)
        del arr, labels, confs, locs, meta
        gc.collect()
    del npz, lab, conf_u8
    gc.collect()
    show_ram(f'{zone} tile freed')

for z in ['pune', 'satara', 'vidarbha']:
    tile_zone_v33(z, LOCAL)
show_ram('Cell 7 End')

```

```

--- Tiling Zone [pune] without full-scene loading ---
[pune tile start] RSS=0.15 GiB | Available=11.40 GiB | Total=12.67 GiB
pune/train: patches=3315 mean change=6.6%
pune/test: patches=817 mean change=9.5%
[pune tile freed] RSS=0.15 GiB | Available=10.41 GiB | Total=12.67 GiB

--- Tiling Zone [satara] without full-scene loading ---
[satara tile start] RSS=0.15 GiB | Available=10.40 GiB | Total=12.67 GiB
satara/train: patches=3349 mean change=10.3%
satara/test: patches=820 mean change=5.6%
[satara tile freed] RSS=0.14 GiB | Available=11.33 GiB | Total=12.67 GiB

--- Tiling Zone [vidarbha] without full-scene loading ---
[vidarbha tile start] RSS=0.14 GiB | Available=11.33 GiB | Total=12.67 GiB
vidarbha/train: patches=0 mean change=0.0%
vidarbha/test: patches=1120 mean change=8.1%
[vidarbha tile freed] RSS=0.18 GiB | Available=11.34 GiB | Total=12.67 GiB
[Cell 7 End] RSS=0.18 GiB | Available=11.34 GiB | Total=12.67 GiB

```

✓ Cell 8: Normalization Statistics (TRAIN AOI ONLY — Zero Leakage)

Computes channel-wise mean and std from **TRAIN AOI ONLY** (Pune West + Satara North). Zero test pixels are included.

```
# ===== CELL 8: NORM STATS (TRAIN AOI ONLY — LOW-MEM CHUNKED STREAMING) =====
import gc
ARRAY_SCALE = 10000.0
SAR_EXTRA_SCALE = 100.0
CHUNK_SIZE = 32 # deliberately conservative on the 12.7 GiB runtime

def compute_norm_stats(zones, data_dir: Path):
    n=0; sm=np.zeros(17,dtype=np.float64); sq=np.zeros(17,dtype=np.float64)
    print('--- Computing Channel Normalization Statistics (Train AOI Only) ---')
    for z in zones:
        path=data_dir/f'{z}_train.npy'
        if not path.exists(): continue
        marr=np.load(path,mmap_mode='r'); total=marr.shape[0]
        print(f' Streaming {z}/train: {total} patches (mmap)...')
        for i in range(0,total,CHUNK_SIZE):
            j=min(i+CHUNK_SIZE,total)
            chunk=marr[i:j].astype(np.float32,copy=True)
            chunk/=np.float32(ARRAY_SCALE)
            for ch in SAR_CH:
                chunk[:,ch,:]*=np.float32(SAR_EXTRA_SCALE)
            sm+=chunk.sum(axis=(0,1,3,4),dtype=np.float64)
            np.square(chunk,out=chunk)
            sq+=chunk.sum(axis=(0,1,3,4),dtype=np.float64)
            n+=chunk.shape[0]*chunk.shape[1]*chunk.shape[3]*chunk.shape[4]
            del chunk
        del marr; gc.collect()
    assert n>0,'No training pixels found!'
    mu=sm/n; sigma=np.sqrt(np.maximum(sq/n-mu**2,1e-12))
    for c in (11,12,16): mu[c]=0.0; sigma[c]=1.0
    stats={'mean':mu.tolist(),'std':sigma.tolist(),'array_scale':ARRAY_SCALE,
           'sar_channels':SAR_CH,'sar_extra_scale':SAR_EXTRA_SCALE,'n_pixels':int(n),
           'source':'TRAIN AOI patches only (pune west + satara north)'}
    json.dump(stats,open(data_dir/'norm_stats_trainonly.json','w'),indent=2)
    channel_labels=S2_BANDS+['NDVI','NDBI','VV','VH','CR','q']
    for i,b in enumerate(channel_labels): print(f' ch{i:2d} {b:5s} mu={mu[i]:+.4f} sd={sigma[i]:.4f}')
    return stats

stats=compute_norm_stats(['pune','satara'],LOCAL)
show_ram('Cell 8 End')
```

```
--- Computing Channel Normalization Statistics (Train AOI Only) ---
Streaming pune/train: 3315 patches (mmap)...
Streaming satara/train: 3349 patches (mmap)...
ch 0 B2    mu=+0.0609 sd=0.0277
ch 1 B3    mu=+0.0837 sd=0.0294
ch 2 B4    mu=+0.1001 sd=0.0430
ch 3 B5    mu=+0.1347 sd=0.0382
ch 4 B6    mu=+0.1862 sd=0.0483
ch 5 B7    mu=+0.2082 sd=0.0571
ch 6 B8    mu=+0.2165 sd=0.0631
ch 7 B8A   mu=+0.2268 sd=0.0630
ch 8 B9    mu=+0.2279 sd=0.0573
ch 9 B11   mu=+0.2251 sd=0.0653
ch10 B12   mu=+0.1612 sd=0.0548
ch11 NDVI  mu=+0.0000 sd=1.0000
ch12 NDBI  mu=+0.0000 sd=1.0000
ch13 VV    mu=-9.8379 sd=4.2250
ch14 VH    mu=-17.2357 sd=4.1598
ch15 CR    mu=-7.3952 sd=2.5022
ch16 q     mu=+0.0000 sd=1.0000
[Cell 8 End] RSS=0.18 GiB | Available=11.31 GiB | Total=12.67 GiB
```

✓ Cell 9: Independent 220-Patch Annotation Pool & QGIS Export — PATCH-WISE I/O

The exact 220-patch allocation is preserved. AlphaEarth/cloud scores are read patch-wise, and QGIS exports read only the current 128x128 patch instead of preloading complete zones.

```
# ===== CELL 9: 220-PATCH POOL + QGIS EXPORT — PATCH-WISE I/O =====
import gc
from collections import Counter

# Scores are cached as Python floats only; no full AlphaEarth scene is cached.
_patch_score_cache = {}
```

```

def patch_band_mean(prefix, row, col, band, scale=1.0):
    key=(prefix,int(row),int(col),int(band),float(scale))
    if key not in _patch_score_cache:
        a=read_mosaic_window(prefix,[band],row,col,PATCH,PATCH,DRIVE_RAW)[0].astype(np.float32)
        _patch_score_cache[key]=float((a/np.float32(scale)).mean())
        del a
    return _patch_score_cache[key]

def build_pool_sample(zone: str, split: str, n_target: int, seed: int=0, exclude_set: set=None):
    if exclude_set is None: exclude_set=set()
    meta_path=LOCAL/f'{zone}_{split}_meta.json'
    if not meta_path.exists(): return []
    meta=json.load(open(meta_path))
    valid=[m for m in meta if (zone,m['row'],m['col']) not in exclude_set]
    if not valid: return []
    prefix=f'{zone}_alphaearth_change'
    score=np.array([patch_band_mean(prefix,m['row'],m['col'],1,10000.0) for m in valid],dtype=np.float32)
    q60,q85,q95=np.percentile(score,[60,85,95])
    strata={'high':(score>=q95,0.40), 'moderate':((score>=q85)&(score<q95),0.30),
            'ambiguous':((score>=q60)&(score<q85),0.20), 'stable':(score<q60,0.10)}
    rng=np.random.default_rng(seed); chosen=[]
    for name,(mask,frac) in strata.items():
        idx=np.flatnonzero(mask); k=min(int(round(n_target*frac)),len(idx))
        for i in rng.choice(idx,size=k,replace=False) if k>0 else []:
            item=valid[int(i)]; chosen.append(**item,'stratum':name,'ae_score':float(score[i]))
            exclude_set.add((zone,item['row'],item['col']))
    rem=n_target-len(chosen)
    if rem>0:
        available=[m for m in valid if (zone,m['row'],m['col']) not in exclude_set]
        if available:
            fill_idx=rng.choice(len(available),size=min(rem,len(available)),replace=False)
            for i in fill_idx:
                item=available[int(i)]; chosen.append(**item,'stratum':'fill',
                    'ae_score':patch_band_mean(prefix,item['row'],item['col'],1,10000.0))
                exclude_set.add((zone,item['row'],item['col']))
    print(f' {zone}/{split} -> selected {len(chosen)} / {n_target} target patches.')
    return chosen

def build_monsoon_pool_sample(n_target=30,seed=50,exclude_set=None):
    if exclude_set is None: exclude_set=set()
    H,W=mosaic_shape('pune_monsoon_T2_optical',DRIVE_RAW)
    valid=[]
    cut=W//2; c_start=cut+BUFFER_PX//2
    for r in range(0,H-PATCH+1,STRIDE_TE):
        for c in range(c_start,W-PATCH+1,STRIDE_TE):
            if ('pune',r,c) in exclude_set: continue
            # q is the only monsoon stratification variable; optical integer tiles are finite.
            valid.append({'zone':'pune','split':'test','row':int(r),'col':int(c)})
    assert valid,'No monsoon candidates found.'
    scores=np.array([patch_band_mean('pune_monsoon_T2_optical',m['row'],m['col'],12,255.0)
        for m in valid],dtype=np.float32)
    assert n_target==30
    target=n_target//3
    rng=np.random.default_rng(seed); chosen=[]
    defs=[('heavy_cloud',scores<=0.40),('moderate_cloud',(scores>0.40)&(scores<=0.80)),
        ('clear_sky',scores>0.80)]
    for name,mask in defs:
        idx=np.flatnonzero(mask)
        assert len(idx)>=target,(f'H2 Protocol Error: {name} has {len(idx)} candidates; need {target}.')
        for i in rng.choice(idx,size=target,replace=False):
            item=valid[int(i)]; chosen.append(**item,'stratum':name,'cloud_q_mean':float(scores[i]))
            exclude_set.add(('pune',item['row'],item['col']))
    counts=Counter(x['stratum'] for x in chosen)
    assert counts=={'heavy_cloud':10,'moderate_cloud':10,'clear_sky':10},counts
    print(f' pune/monsoon -> selected exactly 30 patches: {dict(counts)}')
    return chosen

def export_patch_streaming(zone, row, col, out_dir, blind=False, monsoon=False, label_arr=None):
    out_dir.mkdir(parents=True,exist_ok=True)
    geo_key='pune_monsoon' if monsoon else zone
    prefix1=f'{zone}_monsoon_T1' if monsoon else f'{zone}_T1'
    prefix2=f'{zone}_monsoon_T2' if monsoon else f'{zone}_T2'

```



```

base_tf,crs=mosaic_georef(prefix1+'_optical',DRIVE_RAW)
t1=read_mosaic_window(prefix1+'_optical',[3,2,1,7],row,col,PATCH,PATCH,DRIVE_RAW).astype(np.float32)/np.float32(S2_SCALE)
t2=read_mosaic_window(prefix2+'_optical',[3,2,1,7],row,col,PATCH,PATCH,DRIVE_RAW).astype(np.float32)/np.float32(S2_SCALE)
img=np.concatenate([t1,t2],axis=0)
del t1,t2
tf=rasterio.windows.transform(Window(col,row,PATCH,PATCH),base_tf)
stem=f'{zone}_{row}_{col}'
with rasterio.open(out_dir/f'{stem}.tif','w',driver='GTiff',height=PATCH,width=PATCH,count=8,
                  dtype='float32',crs=crs,transform=tf) as d:
    d.write(img.astype(np.float32)); d.descriptions=('T1_R','T1_G','T1_B','T1_NIR','T2_R','T2_G','T2_B','T2_NIR')
if blind: return
ae=read_mosaic_window(f'{zone}_alphaearth_change',[1],row,col,PATCH,PATCH,DRIVE_RAW)[0].astype(np.float32)/np.float32(100)
with rasterio.open(out_dir/f'{stem}_ae.tif','w',driver='GTiff',height=PATCH,width=PATCH,count=1,
                  dtype='float32',crs=crs,transform=tf) as d: d.write(ae[None])
lab=label_arr[row:row+PATCH,col:col+PATCH]
with rasterio.open(out_dir/f'{stem}_autolabel.tif','w',driver='GTiff',height=PATCH,width=PATCH,count=1,
                  dtype='uint8',crs=crs,transform=tf,nodata=None) as d: d.write(lab[None])

# Exact 220-patch allocation
claimed_patches=set(); pool={}
print('--- Sampling 220-Patch Verification Sets ---')
pool['MH_VAL']=build_pool_sample('pune','train',15,10,claimed_patches)+build_pool_sample('satara','train',15,11,claimed_patches)
pool['MH_ADAPT']=build_pool_sample('pune','train',15,20,claimed_patches)+build_pool_sample('satara','train',15,21,claimed_patches)
pool['MH_TEST_DRY_A']=build_pool_sample('pune','test',40,30,claimed_patches)
pool['MH_TEST_DRY_B']=build_pool_sample('satara','test',40,31,claimed_patches)
pool['MH_TEST_BLIND']=build_pool_sample('pune','test',10,40,claimed_patches)+build_pool_sample('satara','test',10,41,claimed_patches)
pool['MH_TEST_MONSOON']=build_monsoon_pool_sample(30,50,claimed_patches)
pool['MH_TEST_VIDARBHA']=build_pool_sample('vidarbha','test',30,60,claimed_patches)

total_patches=sum(len(v) for v in pool.values())
print(f'\nTotal Patches Sampled: {total_patches} (Target: 220)')
assert total_patches==220
json.dump(pool,open(LOCAL/'annotation_pool.json','w'),indent=2)
_patch_score_cache.clear(); gc.collect()

# Patch-wise QGIS export; only one 128x128 patch is in RAM at a time.
tasks_by_zone={'pune':[],'satara':[],'vidarbha':[],'pune_monsoon':[]}
for split_name,items in pool.items():
    is_blind=('BLIND' in split_name or 'MONSOON' in split_name)
    is_monsoon=('MONSOON' in split_name)
    out_dir=LOCAL/'qgis'/split_name; out_dir.mkdir(parents=True,exist_ok=True)
    for it in items:
        tasks_by_zone['pune_monsoon' if is_monsoon else it['zone']].append({
            'zone':it['zone'],'row':it['row'],'col':it['col'],'out_dir':out_dir,
            'blind':is_blind,'monsoon':is_monsoon})

for geo_key,tasks in tasks_by_zone.items():
    if not tasks: continue
    print(f'\nExporting [{geo_key}] {len(tasks)} patches with patch-wise I/O...')
    lab=None
    zone='pune' if geo_key=='pune_monsoon' else geo_key
    if geo_key!='pune_monsoon':
        lab=np.load(LOCAL/f'{zone}_autolabel_full.npz')['label']
    for t in tqdm(tasks,desc=f'Exporting {geo_key}'):
        export_patch_streaming(t['zone'],t['row'],t['col'],t['out_dir'],blind=t['blind'],monsoon=t['monsoon'],label_arr=lab)
    del lab; gc.collect(); show_ram(f'{geo_key} freed')

print('\nQGIS Verification Stubs successfully generated in /content/proc/qgis/')

```

```

--- Sampling 220-Patch Verification Sets ---
pune/train -> selected 15 / 15 target patches.
satara/train -> selected 15 / 15 target patches.
pune/train -> selected 15 / 15 target patches.
satara/train -> selected 15 / 15 target patches.
pune/test -> selected 40 / 40 target patches.
satara/test -> selected 40 / 40 target patches.
pune/test -> selected 10 / 10 target patches.
satara/test -> selected 10 / 10 target patches.
pune/monsoon -> selected exactly 30 patches: {'heavy_cloud': 10, 'moderate_cloud': 10, 'clear_sky': 10}
vidarbha/test -> selected 30 / 30 target patches.

Total Patches Sampled: 220 (Target: 220)

Exporting [pune] 80 patches with patch-wise I/O...
Exporting pune: 100%                               80/80 [00:07<00:00, 12.86it/s]
[pune freed] RSS=0.18 GiB | Available=11.29 GiB | Total=12.67 GiB

Exporting [satara] 80 patches with patch-wise I/O...
Exporting satara: 100%                             80/80 [00:07<00:00, 13.23it/s]
[satara freed] RSS=0.18 GiB | Available=11.27 GiB | Total=12.67 GiB

Exporting [vidarbha] 30 patches with patch-wise I/O...
Exporting vidarbha: 100%                           30/30 [00:02<00:00, 13.96it/s]
[vidarbha freed] RSS=0.18 GiB | Available=11.27 GiB | Total=12.67 GiB

Exporting [pune_monsoon] 30 patches with patch-wise I/O...
Exporting pune_monsoon: 100%                       30/30 [00:35<00:00, 1.32s/it]
[pune_monsoon freed] RSS=0.18 GiB | Available=11.29 GiB | Total=12.67 GiB

QGIS Verification Stubs successfully generated in /content/proc/qgis/!

```

✓ Cell 10: Drive Archive & Kaggle Staging Preparation

Packs [/content/proc](#) into a single archive on Google Drive:

- `geonexus_v3_processed.tar`

Licensing & Staging Notes:

- Uses license `'other'` with compliant attribution to Copernicus Sentinel open access data and upstream evidence sources (Google Open Buildings Temporal V1 — CC-BY-4.0 or ODbL-1.0, Google Dynamic World CC-BY-4.0, Hansen GFC CC-BY-4.0, JRC Global Surface Water).
- Staging workflow options:
 - **Option A (Direct Colab Staging):** Authenticate and run `!kaggle datasets create -p /content/proc --dir-mode zip`.
 - **Option B (Local Staging):** Extract `geonexus_v3_processed.tar` locally to `data/processed/` and run `python data/stage_kaggle.py --user sumit07125 --dataset geonexus-mh-v3 --new`.

```

# ===== CELL 10: ARCHIVE & PREPARE KAGGLE STAGING =====
print('Creating tar archive in local scratch...')
!cd /content && tar -cf /content/geonexus_v3_processed.tar -C /content proc

print(f'Copying archive to Google Drive ({DRIVE_PROC})...')
!cp /content/geonexus_v3_processed.tar "{DRIVE_PROC}/geonexus_v3_processed.tar"
print('Successfully archived processed dataset to Drive!')

# Dataset metadata for Kaggle staging with compliant mixed-source license
meta = {
    'title': 'Geo-Nexus Maharashtra CD v3.2 Training Data',
    'id': 'sumit07125/geonexus-mh-v3',
    'licenses': [{'name': 'other'}],
    'description': (
        'Multi-modal optical (Sentinel-2) and SAR (Sentinel-1) bi-temporal change detection '
        'training arrays and evidence auto-labels for Maharashtra, India. '
        'Derived from Copernicus Sentinel open access data, with upstream multi-modal '
        'evidence layers from Google Dynamic World (CC-BY-4.0), Google Open Buildings Temporal V1 '
        '(CC-BY-4.0 or ODbL-1.0), Hansen Global Forest Change (CC-BY-4.0), and JRC Global '
        'Surface Water (EC open data / Copernicus programme acknowledgement).'
    )
}

```

```
json.dump(meta, open(LOCAL / 'dataset-metadata.json', 'w'), indent=2)
print('dataset-metadata.json generated. Staging preparation complete.')

# Direct Colab Kaggle Upload (Optional if authenticated via kaggle.json in Colab):
# !mkdir -p ~/.kaggle && cp /content/drive/MyDrive/kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
# !kaggle datasets create -p /content/proc --dir-mode zip
```

```
Creating tar archive in local scratch...
Copying archive to Google Drive (/content/drive/MyDrive/geonexus_v3_processed)...
Successfully archived processed dataset to Drive!
dataset-metadata.json generated. Staging preparation complete.
```